

UIU CSE Lecturer Final Viva Preparation Manual

Complete theory, formulas, exact-question answers, and probable viva Q&A

Prepared for: Ahmmad Nur Swapnil

Purpose: This is not a one-day cram sheet. It is a subject-wise viva manual. Read theory first, then rehearse the related questions after each topic.

Basis: UIU final viva reports you provided, Rafi Vai's suggestions, previous UIU recruitment-experience notes, and the UIU demo/interview subject domains.

How to use: For each subject, first understand the core theory. Then practice answering the Q&A aloud in 30-60 seconds. If paper is given, write formulas clearly before speaking.

0. Interview Strategy and Priority Map

0.1 What the final viva seems to test

From the reports you provided, the final viva is not only about deep research or demo performance. It tests whether you still remember core CSE fundamentals, whether you can reason on paper, and whether you can answer basic questions without overcomplicating them.

- Very high priority: DBMS, DSA, OOP, Algorithms, ML/DL.
- High priority depending on board: DLD, Computer Architecture, Networking/Security, OS.
- Always prepare: thesis, teaching motivation, higher-study plan, CGPA/rank/merit position.
- Question style: short fundamental question -> follow-up -> paper-pen formula or example.
- Expected answer style: direct answer first, then 2-3 lines explanation, then example if needed.

0.2 Ranking of subjects by probability

Priority	Subject	Why it matters	What to master
1	DSA + Algorithms	Repeated in UIU reports and natural CSE viva area	BST, graph, complexity, recursion, MST, search, sorting, DP/greedy
2	OOP	Common final viva favorite-course area	Access modifiers, overloading, overriding, friend, inheritance, diamond problem
3	DBMS	Rafi Vai: one of top three	Keys, joins, normalization, ACID, SQL vs NoSQL, indexing
4	ML/DL	Explicitly reported: regression/classification, CNN, RNN/LSTM	Learning types, CNN layers, activation, overfitting, metrics
5	Networking/Security	Reported: ARP, NAT/PAT, DNS poisoning, IP headers	OSI, switch/router, ARP, DNS, TCP/UDP, NAT, security attacks
6	DLD/Architecture	Reported depending on board	Latches, flip-flops, combinational/sequential, memory hierarchy
7	OS	Usually fundamental CSE area	Process/thread, scheduling, deadlock, semaphore, memory
8	Discrete/Counting	Reported: pigeonhole, balls/cards worst cases	Pigeonhole, combinations, worst-case guarantee logic

0.3 General viva answering template

Direct answer: Give the shortest correct answer first.

Theory: Explain the principle in 2-3 sentences.

Example: Give one small example or draw a diagram if paper is available.

Boundary/Trap: Mention the condition where the answer changes.

Close: Summarize in one sentence.

Q. What if you do not know the exact answer?

A. Say: "I am not fully certain about the exact detail. My understanding is ... I would verify the precise value rather than state something inaccurate." This is better than confidently giving a wrong formula.

Q. How should you handle paper-pen questions?

A. Write the formula or worst-case arrangement first. Then speak. For example, for card guarantee problems, arrange all unwanted cards first, then add the required number of wanted cards.

1. Exact Reported UIU Final Viva Questions - With Direct Answers

This chapter converts the exact reported questions into short model answers. Later chapters explain the full theory behind these answers.

1.1 HR and academic questions

Q. Why do you want to be a teacher?

A. Because I enjoy explaining core CSE ideas clearly and helping students connect theory with implementation. I already have teaching experience as a Lecturer in CSE, and I want to grow as a teacher-researcher who can teach fundamentals, conduct labs, mentor students, and contribute to academic work.

Q. What is your plan for higher studies?

A. I plan to pursue graduate studies in CSE, preferably in areas connected to machine learning, security, software systems, and AI-assisted analysis. Before that, I want to strengthen my teaching and research profile, publish from my current work, and identify a focused research direction.

Q. Your CGPA is X. What is your merit position?

A. Answer with exact rank if known. If asked to compare, be factual and humble: "My CGPA was 3.94/4.00 and my position was among the top students of my batch. I can provide the exact rank if needed."

1.2 Counting, pigeonhole, and worst-case guarantee

Q. 4 red balls, 6 green balls. How many balls should you pick to ensure at least 2 green balls?

A. Worst case, you first pick all 4 red balls. To guarantee 2 green balls, pick 2 more. So answer = $4 + 2 = 6$.

Q. Worst count for taking 2 aces from a deck of 52 cards?

A. There are 4 aces and 48 non-aces. Worst case, you first pick all 48 non-aces, then need 2 aces. Answer = $48 + 2 = 50$.

Q. Worst case, how many cards must you pick to ensure 3 red cards?

A. There are 26 black cards and 26 red cards. Worst case, first pick all 26 black cards, then pick 3 red cards. Answer = $26 + 3 = 29$.

Q. What is the pigeonhole principle?

A. If $n+1$ or more objects are placed into n boxes, at least one box contains at least two objects. General form: if N objects go into k boxes, some box has at least $\text{ceil}(N/k)$ objects.

1.3 Matrix address calculation

Q. How to find address of an integer in a row-major matrix when base address is given?

A. For $A[m][n]$ stored row-major, address of $A[i][j] = \text{base} + ((i - \text{lower_row}) * \text{number_of_columns} + (j - \text{lower_col})) * \text{element_size}$. In C with 0-based indexing: $\text{address} = \text{base} + (i * \text{columns} + j) * \text{sizeof}(\text{element})$.

Q. How is column-major different?

A. Column-major stores columns contiguously. Formula: $\text{base} + ((j - \text{lower_col}) * \text{number_of_rows} + (i - \text{lower_row})) * \text{element_size}$.

1.4 ML/DL exact questions

Q. Regression vs classification with example?

A. Regression predicts a continuous numeric value, such as house price or temperature. Classification predicts a discrete class, such as spam/not spam or disease/no disease.

Q. Semi-supervised learning: how will you train if some data are labeled and some are unlabeled?

A. Train first on labeled data, use the model to generate pseudo-labels for confident unlabeled samples, retrain using both labeled and pseudo-labeled data, and validate on a held-out labeled set. Other methods include consistency regularization and graph-based label propagation.

Q. What does a convolution layer do?

A. It applies learnable filters/kernels over local regions of an input image or feature map to extract local patterns such as edges, corners, textures, and higher-level features.

Q. What is pooling layer used for?

A. Pooling reduces spatial dimension, lowers computation, and makes features more robust to small translations. Max pooling keeps the strongest activation; average pooling takes the average.

Q. Usage of RNN and LSTM?

A. RNNs process sequential data such as text, speech, and time series. LSTMs are improved RNNs with gates that help preserve long-term dependencies and reduce vanishing-gradient problems.

Q. What is the difference between machine learning and deep learning?

A. Machine learning is a broad field where models learn patterns from data. Deep learning is a subset of ML using multi-layer neural networks that can learn hierarchical features, often requiring more data and computation.

Q. What activation function is usually used in deep learning?

A. ReLU is commonly used in hidden layers because it is simple, efficient, and reduces vanishing-gradient issues compared to sigmoid/tanh. Sigmoid is often used for binary output, softmax for multiclass output.

Q. Example of clustering?

A. Grouping customers based on purchasing behavior, grouping similar news articles, segmenting images, or clustering students by learning patterns. K-means is a common clustering algorithm.

1.5 Search and algorithms exact questions

Q. What is depth-limited search?

A. Depth-limited search is DFS with a maximum depth limit. It avoids going infinitely deep but may miss a solution if the limit is too small.

Q. Given social network graph, which MST algorithm is better?

A. For dense social-network-like graphs, Prim with adjacency matrix can be $O(V^2)$, often better than sorting all edges as in Kruskal. With adjacency list and binary heap, Prim is $O(E \log V)$. Kruskal is $O(E \log E)$, often good for sparse graphs.

Q. Find quotient and remainder of a divided by b without division or modulus.

A. Repeatedly subtract b from a until $a < b$. The count is quotient and remaining a is remainder. For efficiency, use bit shifting/doubling. Handle $b=0$ as invalid and handle sign separately.

Q. Find GCD using recursion.

A. Use Euclid algorithm: $\text{gcd}(a,b) = \text{gcd}(b, a \bmod b)$, base case $\text{gcd}(a,0)=a$. If modulus is not allowed, use repeated subtraction, but modulus version is standard.

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

1.6 BST and trees exact questions

Q. Given a balanced/perfect binary tree of height h, what is the total number of nodes?

A. Clarify height convention. If root height = 0, perfect binary tree nodes = $2^{(h+1)} - 1$. If root level/height is counted as 1, nodes = $2^h - 1$.

Q. Why was "about 2^h " not enough?

A. Because viva often expects exact calculation. Sum of nodes by level = $1 + 2 + 4 + \dots + 2^h = 2^{(h+1)} - 1$ when root height is 0.

1.7 OOP exact questions

Q. Difference between public, private, and protected in a class?

A. Public members are accessible from outside the class. Private members are accessible only inside the class. Protected members are accessible inside the class and derived classes, but not general outside code.

Q. Can private variables be accessed by other classes?

A. Normally no. They should be accessed through public methods/getters/setters if allowed. In C++, friend functions/classes can access private members if explicitly declared. Some languages may allow reflection, but that breaks normal encapsulation rules.

Q. What is function overloading?

A. Same function name with different parameter lists in the same scope. It is compile-time polymorphism.

Q. Why cannot we overload only by return type?

A. Because when a function is called, the compiler usually resolves overloads from function name and argument list. Return type alone may be ambiguous, especially if the return value is ignored.

Q. Corner case of multiple inheritance?

A. The diamond problem: one class inherits from two classes that both inherit from a common base, causing ambiguity or duplicate base subobjects. In C++, virtual inheritance can solve this.

1.8 Networking/security exact questions

Q. PAT vs NAT?

A. NAT maps private IP addresses to public IP addresses. PAT is a type of NAT that uses port numbers so many private hosts can share one public IP. PAT is also called NAT overload.

Q. What is ARP? Explain process.

A. ARP means Address Resolution Protocol. It maps an IPv4 address to a MAC address inside a local network. A host broadcasts: "Who has IP X?" The owner replies with its MAC address. The sender caches the mapping.

Q. What layers do switch and router operate on?

A. Switch usually operates at Layer 2 Data Link layer using MAC addresses. Router operates at Layer 3 Network layer using IP addresses. Some modern multilayer switches can also do Layer 3 functions.

Q. Header size of IPv4 and IPv6?

A. IPv4 header is minimum 20 bytes and maximum 60 bytes because of options. IPv6 fixed base header is 40 bytes; extension headers are separate.

Q. What problem will arise due to DNS poisoning?

A. DNS poisoning can redirect users from a legitimate domain to a malicious IP address, enabling phishing, malware delivery, man-in-the-middle attacks, or service disruption.

1.9 DLD and computer architecture exact questions

Q. What are memory elements used in computers?

A. Latch, flip-flop, register, cache memory, RAM, ROM, secondary storage. At digital logic level, latches and flip-flops store bits; registers store groups of bits; memory arrays store larger data.

Q. Describe different latches.

A. Common latches include SR latch, D latch, JK latch, and T latch. SR stores set/reset state, D stores input data when enabled, JK avoids invalid SR combination by toggling when both inputs are 1, and T toggles state when enabled.

Q. Why is SR latch not commonly used?

A. Because the condition $S=1$ and $R=1$ can be invalid or forbidden depending on implementation, leading to undefined or unstable behavior. D/JK/flip-flop designs are safer for practical circuits.

Q. Combinational vs sequential circuits?

A. Combinational circuit output depends only on current inputs, such as adder or multiplexer. Sequential circuit output depends on current inputs and previous state, such as flip-flop, counter, or register.

2. HR, Teaching Motivation, Higher Studies, and Thesis Defense

2.1 Why do you want to be a teacher? Full theory behind answer

A lecturer interview is not only testing whether you know CSE. It tests whether you understand the responsibility of teaching. A strong answer should contain motivation, experience, teaching philosophy, and long-term growth.

- Motivation: You enjoy simplifying difficult CSE concepts.
- Experience: You are already teaching theory and sessional core CSE courses.
- Teaching philosophy: Connect concept, code, and real example.
- Future contribution: Teach, mentor, create materials, supervise projects, and grow as a researcher.

Q. Why teacher, not software industry?

A. Industry solves product problems directly, but teaching creates long-term impact by developing many students. I like software and research, but teaching allows me to combine technical depth, mentoring, and academic contribution.

Q. What is good teaching?

A. Good teaching means correct content, clear explanation, proper pacing, examples, checking student understanding, and connecting theory to practical implementation.

Q. How do you handle weak students?

A. I break the topic into smaller steps, use simple examples, repeat core intuition, and give guided practice before independent tasks. Weak students often improve when the first barrier is removed.

Q. How do you handle strong students?

A. Give them extension problems, deeper proofs, optimization variants, and project/research directions so they remain challenged.

2.2 Higher studies answer

They may ask higher studies to check seriousness. Do not sound like you will leave immediately and not care about the institution. Also do not sound like you have no plan. Balanced answer: you want to grow academically while contributing responsibly.

Q. What is your plan for higher studies?

A. I plan to pursue graduate studies in CSE in a focused area such as machine learning, security, or AI-assisted software/security analysis. Before that, I want to strengthen my teaching, publish and extend my current research, and build a clear research direction. If I join an institution, I will also take my academic responsibilities seriously.

Q. Will higher studies conflict with teaching?

A. Not necessarily. Graduate preparation and teaching can complement each other because teaching strengthens fundamentals and research strengthens depth. I would follow institutional policy and maintain responsibilities properly.

2.3 Thesis preparation: LLM-Guided Security Analysis of Matter Protocol

Prepare a 30-second, 1-minute, and 3-minute version. Most panels do not want a full thesis defense. They want to know whether you understand your own work and can explain it simply.

30-second version

My thesis is titled "LLM-Guided Security Analysis of Matter Protocol." The goal was to analyze the Matter IoT protocol using LLM-assisted reasoning and practical validation. We studied protocol behavior and identified potential security weaknesses. The work combines protocol security, AI-assisted analysis, and practical IoT security evaluation.

1-minute version

Matter is an IoT protocol intended to provide interoperable smart-home communication. My thesis explored whether large language models can assist security analysis of such a protocol. The work involved reading protocol specifications, extracting security-relevant behaviors, asking LLMs to reason about possible weaknesses, and validating findings carefully instead of trusting the model blindly. The contribution is not just using an LLM, but designing a workflow where LLM suggestions are checked against protocol logic and implementation behavior.

Probable thesis questions

Q. Why did you use LLMs for security analysis?

A. LLMs can help read long specifications, generate hypotheses, and connect dispersed protocol details. But they can hallucinate, so the important part is to use them as assistants and validate findings systematically.

Q. What is the limitation of your approach?

A. LLMs may hallucinate, miss low-level implementation details, or overgeneralize. Therefore, every claim must be validated using specification evidence, implementation behavior, or experiments.

Q. What is Matter protocol?

A. Matter is a smart-home IoT connectivity standard designed for interoperability among devices and ecosystems. It uses modern security mechanisms, but complex protocols still require careful security analysis.

Q. What is responsible disclosure?

A. Responsible disclosure means reporting vulnerabilities to the responsible organization privately first, giving them time to investigate and mitigate before public disclosure.

Q. How is your thesis related to teaching?

A. It strengthened my ability to read complex technical documents, reason systematically, and explain security concepts clearly. These are useful for teaching OS, networking, security, software engineering, and AI-related courses.

3. Object-Oriented Programming - Full Theory and Viva Q&A

3.1 Class, object, and encapsulation

A class is a blueprint that defines data and behavior. An object is an instance of a class. Encapsulation means keeping data and related methods together while controlling access to internal data using access modifiers. It improves maintainability because internal representation can change without changing external use.

```
class Student {
private:
    int id;
    double cgpa;
public:
    void setCgpa(double c) { if (c >= 0 && c <= 4) cgpa = c; }
    double getCgpa() { return cgpa; }
};
```

Q. Why keep data private?

A. To protect invariants and prevent invalid modification. For example, CGPA should not become 10.0 in a 4.0 scale.

Q. Is getter/setter always good?

A. Not always. Too many trivial getters/setters may expose internal design. Better is to provide meaningful operations when possible.

3.2 Access modifiers: public, private, protected

Modifier	Accessible from same class	Derived class	Outside object
public	Yes	Yes	Yes
private	Yes	No directly	No
protected	Yes	Yes	No

Q. Can private data be accessed by child class?

A. No, not directly. A child class can access protected members, and it may use public/protected methods of the parent to interact with private data.

Q. Can private data be accessed anyway?

A. Normally no. In C++, friend functions/classes can access if explicitly allowed. Public getters can expose read access. Low-level hacks or reflection are possible in some languages, but they violate the normal abstraction model.

3.3 Constructor, destructor, static members

A constructor initializes an object when it is created. A destructor releases resources when an object is destroyed. Static members belong to the class, not to individual objects.

```
class Counter {
public:
    static int count;
    Counter() { count++; }
    ~Counter() { count--; }
};
```

Q. Can constructor have return type?

A. No. Constructor has no return type, not even void.

Q. Can constructor be overloaded?

A. Yes. Multiple constructors may exist with different parameter lists.

Q. When is destructor important?

A. When the object owns resources such as dynamic memory, file handles, sockets, or locks.

3.4 Inheritance and types

Inheritance allows one class to reuse and extend another class. It represents an "is-a" relationship. For example, Circle is a Shape. Inheritance supports code reuse and polymorphism, but overuse can create tight coupling.

- Single inheritance: one base class.
- Multiple inheritance: more than one base class, supported in C++ but not directly in Java classes.
- Multilevel inheritance: A -> B -> C.
- Hierarchical inheritance: multiple classes inherit from one base.
- Hybrid inheritance: combination of forms.

Q. Composition vs inheritance?

A. Inheritance means is-a; composition means has-a. Prefer composition when reuse does not require subtype relationship.

3.5 Polymorphism, overloading, overriding, virtual function

Polymorphism means one interface can have many implementations. Compile-time polymorphism includes function overloading and operator overloading. Runtime polymorphism happens through overriding and virtual functions.

```
class Shape {
public:
    virtual void draw() { cout << "Shape"; }
};
class Circle : public Shape {
public:
    void draw() override { cout << "Circle"; }
};
Shape* s = new Circle();
s->draw(); // Circle, because draw is virtual
```

Concept	Meaning	Binding
Overloading	Same function name, different parameters	Compile-time
Overriding	Child redefines parent method with same signature	Runtime if virtual/dynamic dispatch
Virtual function	Base method resolved using actual object type	Runtime
Abstract class	Class with at least one pure virtual method in C++	Design level

Q. Why cannot function overloading depend only on return type?

A. Because the compiler resolves a call mainly from function name and arguments. If two functions differ only by return type, a call like f(5); is ambiguous when return value is ignored.

Q. What is dynamic binding?

A. The method call is resolved at runtime based on actual object type rather than pointer/reference type.

Q. What is pure virtual function?

A. A function declared as =0 in C++. It makes the class abstract and forces derived classes to implement it.

3.6 Friend function

A friend function in C++ is not a member of the class but is allowed to access private and protected members because the class explicitly grants friendship.

```
class Box {
private:
    int width;
public:
    Box(int w) : width(w) {}
    friend void printWidth(Box b);
};
void printWidth(Box b) { cout << b.width; }
```

Q. Does friend function violate encapsulation?

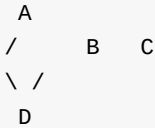
A. It can reduce encapsulation if overused. But it is useful for operator overloading or functions that need controlled access to internals.

Q. Is friendship inherited?

A. No. Friendship is not inherited and not transitive.

3.7 Multiple inheritance and diamond problem

The diamond problem occurs when a class inherits from two classes that both inherit from a common base. This can create duplicate base-class subobjects or ambiguity about which base member to use.



In C++, virtual inheritance can make B and C share a single A subobject inside D.

Q. What is the solution to the diamond problem?

A. In C++, use virtual inheritance. In Java, classes do not support multiple class inheritance, but interfaces can be used with rules for default methods.

Q. Why does Java avoid multiple class inheritance?

A. To avoid ambiguity and complexity such as the diamond problem. Java supports multiple inheritance of type through interfaces.

3.8 OOP rapid Q&A

Q. Difference between abstraction and encapsulation?

A. Abstraction hides unnecessary details and shows essential features. Encapsulation bundles data and methods and restricts access.

Q. Interface vs abstract class?

A. Interface defines a contract. Abstract class can contain shared state and partial implementation. In modern languages details vary, but the design idea remains contract vs partial base implementation.

Q. Can static method be virtual?

A. In C++, static member functions cannot be virtual because virtual dispatch requires an object.

Q. What is method overriding condition?

A. Same method signature in derived class as base class method; for runtime polymorphism in C++, base method should be virtual.

Q. What is operator overloading?

A. Giving operators like + or << user-defined behavior for custom types.

Q. What is coupling?

A. Degree of dependency between modules/classes. Lower coupling is usually better.

Q. What is cohesion?

A. How strongly related responsibilities inside a module are. High cohesion is usually better.

4. Data Structures - Full Theory and Viva Q&A

4.1 Complexity basics

Time complexity estimates how running time grows with input size. Space complexity estimates memory growth. In viva, always mention best/average/worst when relevant.

Notation	Meaning	Example
$O(1)$	Constant time	Access array element by index
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Scan an array
$O(n \log n)$	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Nested loops over n
$O(2^n)$	Exponential	Brute force subsets
$O(n!)$	Factorial	Brute force permutations

Q. What is worst-case complexity?

A. Maximum time taken among all inputs of size n .

Q. Why ignore constants in Big-O?

A. Because Big-O describes asymptotic growth as n becomes large. Constants matter in practice but not in asymptotic classification.

4.2 Array vs linked list

Array stores elements in contiguous memory. Linked list stores nodes, each containing data and one or more pointers. Arrays are better for random access; linked lists are better for frequent insertion/deletion when node position is known.

Operation	Array	Linked list
Access by index	$O(1)$	$O(n)$
Search by value	$O(n)$	$O(n)$
Insert at beginning	$O(n)$	$O(1)$
Insert after known node	$O(n)$ if shifting needed	$O(1)$
Memory	Contiguous, less overhead	Non-contiguous, pointer overhead

Q. Why is array access $O(1)$?

A. Because address can be computed directly: $\text{base} + \text{index} * \text{element_size}$.

Q. Why linked list random access is $O(n)$?

A. Because nodes are not contiguous, so we must traverse from the head.

4.3 Stack and queue

Stack follows LIFO: last in, first out. Queue follows FIFO: first in, first out. Stack is used in recursion, expression evaluation, undo operations. Queue is used in BFS, scheduling, buffering.

- Stack operations: push, pop, top/peek.
- Queue operations: enqueue, dequeue, front.
- Circular queue avoids wasted space in array implementation.

Q. Where is stack used in recursion?

A. Each recursive call creates an activation record on the call stack containing local variables, parameters, and return address.

Q. What is queue used for in BFS?

A. BFS explores nodes level by level, so it uses a queue to process earlier discovered nodes first.

4.4 Trees and binary trees

A tree is a connected acyclic graph. A binary tree is a tree where each node has at most two children. A full binary tree has every node with 0 or 2 children. A complete binary tree fills all levels except possibly last, from left to right. A perfect binary tree has all internal nodes with two children and all leaves at same level.

Tree type	Definition
Full	Every node has 0 or 2 children
Complete	All levels full except maybe last; last filled left to right
Perfect	All levels fully filled
Balanced	Height kept $O(\log n)$, exact definition depends on structure
BST	Left subtree keys smaller, right subtree keys larger

Q. Nodes in perfect binary tree of height h?

A. If root height = 0, nodes = $2^{(h+1)}-1$. If root level = 1, nodes = 2^h-1 .

Q. Height of balanced BST with n nodes?

A. $O(\log n)$. More precisely, for perfect tree with height h and n nodes: $n = 2^{(h+1)}-1$, so $h = \log_2(n+1)-1$.

4.5 Binary Search Tree

BST maintains ordered property: all keys in left subtree are smaller, all keys in right subtree are larger. Inorder traversal gives sorted order. Average search/insert/delete time is $O(\log n)$ if balanced, but $O(n)$ if skewed.

```
search(root, key):
    if root == NULL or root->key == key: return root
    if key < root->key: search(root->left, key)
    else: search(root->right, key)
```

Q. Why can BST degrade to $O(n)$?

A. If inserted keys are sorted, the tree can become like a linked list.

Q. How to keep BST balanced?

A. Use self-balancing trees such as AVL tree or Red-Black tree.

Q. What is inorder traversal of BST?

A. It visits nodes in sorted order: left, root, right.

4.6 Heap and priority queue

A binary heap is a complete binary tree satisfying heap property. In max-heap, parent $\geq$ children. In min-heap, parent $\leq$ children. It is commonly stored in array.

```
For 0-based heap array:
parent(i) = (i-1)/2
left(i) = 2*i + 1
right(i) = 2*i + 2
```

Q. Insert complexity in heap?

A. $O(\log n)$, because new element may bubble up along tree height.

Q. Extract-min/max complexity?

A. $O(\log n)$, because root is removed and heapify-down occurs.

4.7 Hash table

Hash table stores key-value pairs using a hash function. Average search/insert/delete is $O(1)$, but worst case can be $O(n)$ if many collisions occur.

- Collision resolution: chaining or open addressing.
- Load factor = $\text{number_of_elements} / \text{table_size}$.
- Rehashing reduces collision when load factor becomes high.

Q. Is hash search always $O(1)$?

A. No. Average expected $O(1)$, worst $O(n)$ if collisions are severe.

Q. What makes a good hash function?

A. It distributes keys uniformly, is fast to compute, and minimizes collisions.

4.8 Graph types and representations

A graph $G=(V,E)$ has vertices and edges. It can be directed/undirected, weighted/unweighted, connected/disconnected, cyclic/acyclic, simple/multigraph, sparse/dense.

Representation	Space	Good for
Adjacency matrix	$O(V^2)$	Dense graphs, $O(1)$ edge check
Adjacency list	$O(V+E)$	Sparse graphs, efficient traversal
Edge list	$O(E)$	Kruskal MST and simple edge processing

Q. What is dense graph?

A. A graph where number of edges is close to V^2 . For undirected simple graph, maximum edges = $V(V-1)/2$.

Q. What is sparse graph?

A. A graph where E is much smaller than V^2 , often close to $O(V)$.

Q. Social network graph dense or sparse?

A. Real social networks are often sparse globally, but if the question says dense or heavily connected, Prim with adjacency matrix can be justified. Answer carefully based on given condition.

4.9 DSA rapid Q&A

Q. How to find middle node of linked list without counting?

A. Use slow and fast pointers. Slow moves one step, fast moves two. When fast reaches end, slow is at middle.

Q. Detect cycle in linked list?

A. Floyd cycle detection: slow and fast pointers. If they meet, cycle exists.

Q. Stack vs heap memory?

A. Stack stores function calls/local automatic variables; heap stores dynamically allocated memory controlled by programmer/runtime.

Q. B-tree vs BST?

A. B-tree is multiway balanced tree optimized for disk/database indexes; BST is binary and usually memory-based.

Q. What is AVL tree?

A. A self-balancing BST where balance factor of each node is -1, 0, or +1.

Q. What is trie?

A. A prefix tree used for strings, dictionaries, autocomplete, IP routing.

5. Algorithms - Full Theory and Viva Q&A

5.1 Searching

Linear search scans elements one by one and works on unsorted data. Binary search requires sorted data and repeatedly halves the search interval.

Algorithm	Requirement	Time
Linear search	None	$O(n)$
Binary search	Sorted array/random access	$O(\log n)$
Hash lookup	Hash table	Average $O(1)$, worst $O(n)$

Q. Why binary search is $O(\log n)$?

A. Each step halves the search space. After k steps, remaining size is $n/2^k$. Stop when $n/2^k = 1$, so $k = \log_2 n$.

Q. Can binary search be used on linked list?

A. Not efficiently in normal linked list because accessing middle is $O(n)$.

5.2 Sorting

Sort	Best	Average	Worst	Stable?	Core idea
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Swap adjacent inversions
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Select minimum each pass
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Insert into sorted prefix
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	Divide and merge
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No typical	Partition by pivot
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No	Use heap

Q. Why quicksort worst case $O(n^2)$?

A. If pivot repeatedly splits very unevenly, e.g., smallest/largest element each time.

Q. Why merge sort needs extra memory?

A. Merging two sorted halves usually requires temporary arrays, giving $O(n)$ extra space.

5.3 BFS, DFS, and depth-limited search

BFS uses a queue and explores level by level. DFS uses stack/recursion and explores deeply before backtracking. Depth-limited search is DFS with a depth bound.

Search	Data structure	Complete?	Optimal?	Use
BFS	Queue	Yes for finite branching	Yes for unweighted/equal cost	Shortest path in unweighted graph
DFS	Stack/recursion	No for infinite depth	No	Traversal, topological sort
DLS	Stack with limit	Only if solution within limit	No	Avoid infinite depth
IDS	Repeated DLS	Yes for finite branching	Yes for unit cost	AI search when depth unknown

Q. What is depth-limited search?

A. DFS with a maximum depth. It returns cutoff if limit is reached before solution.

Q. What is iterative deepening search?

A. Run depth-limited search with limits $0, 1, 2, \dots$ until solution found. It combines BFS optimality for unit costs with DFS low memory.

5.4 Shortest path algorithms

Algorithm	Handles negative edge?	Detects negative cycle?	Use
-----------	------------------------	-------------------------	-----

BFS	No weights or equal weights only	No	Unweighted shortest path
Dijkstra	No negative weights	No	Non-negative weighted shortest path
Bellman-Ford	Yes	Yes	Negative edges
Floyd-Warshall	Yes, no negative cycle	Can detect	All-pairs shortest path

Q. Why Dijkstra fails with negative edges?

A. Dijkstra finalizes a node assuming no later path can reduce its distance. A negative edge can reduce a finalized distance later, breaking that assumption.

Q. What algorithm for negative weights?

A. Bellman-Ford for single-source shortest path. It also detects negative cycles.

5.5 Minimum Spanning Tree: Prim vs Kruskal

A spanning tree connects all vertices without cycles. An MST is a spanning tree with minimum total edge weight. MST applies to connected, undirected, weighted graphs.

Algorithm	Core idea	Good for	Complexity
Kruskal	Sort edges, add smallest non-cycle edge using DSU	Sparse graphs / edge list	$O(E \log E)$
Prim matrix	Grow tree from a start vertex	Dense graphs	$O(V^2)$
Prim heap	Use priority queue of crossing edges	Sparse/dense with adjacency list	$O(E \log V)$

Q. Given dense social network graph, which MST algorithm?

A. If dense and adjacency matrix is used, Prim can be $O(V^2)$, which is good for dense graphs. Kruskal requires sorting many edges, $O(E \log E)$, which is heavy when E is large.

Q. Does MST make sense for directed graph?

A. Standard MST is for undirected graphs. For directed graphs, related concept is minimum spanning arborescence.

5.6 Greedy vs dynamic programming

Greedy chooses the locally best option at each step. DP solves overlapping subproblems and stores results. Both need optimal substructure, but greedy also needs greedy-choice property.

Feature	Greedy	DP
Decision	Local best choice	Combines subproblem solutions
Memory	Usually low	Memo/table often needed
Examples	Kruskal, Prim, Huffman, fractional knapsack	0/1 knapsack, LCS, edit distance
Risk	May fail without greedy-choice property	Can be slower but more exhaustive

Q. Fractional vs 0/1 knapsack?

A. Fractional knapsack works greedily by value/weight ratio. 0/1 knapsack generally needs DP because items cannot be split.

Q. Recursion vs DP?

A. Recursion is a programming technique. DP is an algorithmic technique that may use recursion with memoization or iterative tabulation.

5.7 Quotient/remainder without division/modulus

Naive solution: repeated subtraction. For positive a,b: subtract b until $a < b$. Count subtractions as quotient; remaining a as remainder. Efficient solution: subtract largest doubled multiples of b using bit shifts.

```

quotient = 0
while a >= b:
    a = a - b
    quotient = quotient + 1

```

```
remainder = a
```

Q. Corner cases?

A. $b=0$ invalid. Negative numbers require sign handling. If $a < b$, $\text{quotient}=0$ and $\text{remainder}=a$.

5.8 Recursion and GCD

Recursion solves a problem by reducing it to smaller versions of the same problem. It needs a base case and recursive case. Euclid algorithm computes GCD efficiently.

```
gcd(a, b):  
    if b == 0: return a  
    return gcd(b, a mod b)
```

Q. Why Euclid works?

A. Because $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$. Any common divisor of a and b also divides $a - qb$, which is the remainder.

Q. Time complexity of Euclid algorithm?

A. $O(\log \min(a, b))$ approximately, more precisely proportional to number of digits for standard arithmetic model.

6. Discrete Mathematics, Counting, and Pigeonhole

6.1 Proposition, implication, and quantifier

A proposition is a statement that is either true or false. An implication $p \rightarrow q$ is false only when p is true and q is false. Quantifiers express statements over a domain: forall means for every, exists means at least one.

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Q. Contrapositive of $p \rightarrow q$?

A. not $q \rightarrow$ not p . It is logically equivalent to the original implication.

Q. Converse of $p \rightarrow q$?

A. $q \rightarrow p$. It is not generally equivalent.

6.2 Pigeonhole principle

Basic pigeonhole principle: If $n+1$ objects are placed into n boxes, at least one box has at least two objects. Generalized: if N objects are placed into k boxes, at least one box has at least $\lceil N/k \rceil$ objects.

Q. Among 13 people, prove two are born in same month.

A. There are 12 months as boxes and 13 people as objects. By pigeonhole, at least one month contains at least two people.

Q. How many integers must be chosen from 1 to 10 to ensure two have same parity?

A. There are 2 parity boxes: even and odd. To ensure two same parity, pick 3 integers.

6.3 Worst-case guarantee technique

Many viva counting questions ask: "How many must you pick to ensure X ?" Strategy: first pick as many unwanted items as possible, then add the required wanted items. This is worst-case reasoning.

Problem	Worst case first	Then needed	Answer
Ensure 2 green from 4 red, 6 green	4 red	2 green	6
Ensure 2 aces from deck	48 non-aces	2 aces	50
Ensure 3 red cards	26 black	3 red	29
Ensure at least 1 ace	48 non-aces	1 ace	49
Ensure all 4 suits	Take 13 from three suits = 39	1 from fourth suit	40

Q. Why for 3 red cards answer is 29, not 3?

A. Because the question asks worst-case guarantee. You could first draw all 26 black cards, so only after 29 cards are you guaranteed 3 red cards.

6.4 Permutation and combination

Permutation counts arrangements where order matters. Combination counts selections where order does not matter.

Permutation: $P(n, r) = n! / (n-r)!$

Combination: $C(n, r) = n! / (r!(n-r)!)$

Q. When to use combination?

A. When selecting items and order does not matter, e.g., choosing 3 students from 10.

Q. When to use permutation?

A. When arrangement/order matters, e.g., arranging 3 students in 3 seats.

6.5 Induction

Mathematical induction proves a statement for all natural numbers. Step 1: prove base case. Step 2: assume true for k . Step 3: prove true for $k+1$.

Q. Prove $1+2+\dots+n = n(n+1)/2$ by induction.

A. Base $n=1$ true. Assume sum to $k = k(k+1)/2$. Then sum to $k+1 = k(k+1)/2 + (k+1) = (k+1)(k+2)/2$, so true.

7. Database Management Systems - Full Theory and Viva Q&A

7.1 Basic DBMS concepts

A database is an organized collection of data. A DBMS is software that manages storage, retrieval, update, concurrency, security, and recovery. Relational DBMS stores data in tables with rows and columns.

Term	Meaning
Relation	Table
Tuple	Row
Attribute	Column
Domain	Allowed values of an attribute
Schema	Structure/design
Instance	Actual data at a time

Q. Why use DBMS instead of files?

A. DBMS provides data independence, query language, concurrency control, security, integrity constraints, transactions, backup and recovery.

7.2 Keys

Key	Meaning
Super key	Any attribute set that uniquely identifies a row
Candidate key	Minimal super key
Primary key	Chosen candidate key
Alternate key	Candidate key not chosen as primary
Foreign key	Attribute referencing primary/candidate key of another table
Composite key	Key using multiple attributes

Q. Primary key vs foreign key?

A. Primary key uniquely identifies rows in its own table. Foreign key references a key in another table and enforces relationship/integrity.

Q. Can primary key be NULL?

A. No. Primary key must be unique and not NULL.

Q. Can foreign key be NULL?

A. Yes, if relationship is optional and constraint allows it.

7.3 ER model and relationships

ER model represents entities, attributes, and relationships. Relationship cardinality can be one-to-one, one-to-many, or many-to-many. Many-to-many is usually converted into a separate junction table in relational design.

Q. How to convert many-to-many relationship to tables?

A. Create a new relation containing foreign keys of the participating entities, often together forming a composite primary key.

7.4 Normalization

Normalization organizes data to reduce redundancy and anomalies. It is based on functional dependencies. However, too much normalization can increase join cost; sometimes denormalization is used for performance.

Normal form	Condition	Problem solved
1NF	Atomic values, no repeating groups	Repeating/multivalued columns
2NF	1NF + no partial dependency on part of composite key	Partial dependency
3NF	2NF + no transitive dependency from key to non-key via non-key	Transitive dependency
BCNF	For every FD $X \rightarrow Y$, X is a super key	Stronger dependency anomalies

Q. What is partial dependency?

A. A non-prime attribute depends on part of a composite candidate key, not the whole key.

Q. What is transitive dependency?

A. A non-key attribute depends on another non-key attribute, e.g., student_id -> dept_id and dept_id -> dept_name.

Q. 3NF vs BCNF?

A. BCNF is stricter. In BCNF, every determinant must be a candidate/super key. Some 3NF relations are not BCNF.

7.5 SQL basics and joins

SQL has DDL for defining schema, DML for manipulating data, DCL for permissions, and TCL for transactions.

Category	Commands
DDL	CREATE, ALTER, DROP, TRUNCATE
DML	SELECT, INSERT, UPDATE, DELETE
DCL	GRANT, REVOKE
TCL	COMMIT, ROLLBACK, SAVEPOINT

Join	Meaning
INNER JOIN	Rows that match in both tables
LEFT JOIN	All left rows + matched right rows
RIGHT JOIN	All right rows + matched left rows
FULL OUTER JOIN	All rows from both sides, matched where possible
CROSS JOIN	Cartesian product

Q. WHERE vs HAVING?

A. WHERE filters rows before grouping. HAVING filters groups after GROUP BY.

Q. DELETE vs TRUNCATE?

A. DELETE removes selected rows and can use WHERE; TRUNCATE removes all rows faster and is DDL in many systems.

7.6 ACID and transactions

A transaction is a logical unit of work. ACID properties ensure reliability.

Property	Meaning	Example
Atomicity	All or nothing	Money transfer debit and credit both happen or none
Consistency	Preserves constraints	Balance cannot violate rule
Isolation	Concurrent transactions appear controlled	Two users cannot book same seat incorrectly
Durability	Committed data survives crash	Committed payment remains after power loss

Q. What is dirty read?

A. Reading uncommitted data from another transaction.

Q. What is lost update?

A. Two transactions update same data and one update overwrites the other.

Q. How DBMS ensures isolation?

A. Using locks, timestamps, MVCC, and isolation levels.

7.7 Indexing

An index is an auxiliary data structure that speeds up retrieval. Commonly B+ trees or hash indexes are used. Index improves search but costs extra storage and slows insert/update/delete because index must be maintained.

Q. Why database index often uses B+ tree?

A. B+ tree is balanced and disk-friendly. It supports range queries efficiently and keeps data sorted in leaf level.

Q. Clustered vs non-clustered index?

A. Clustered index determines physical/logical ordering of table data. Non-clustered index is separate and points to data rows.

7.8 SQL vs NoSQL

Feature	SQL	NoSQL
Model	Relational tables	Document/key-value/column/graph
Schema	Fixed/schema-first	Flexible/schema-less or schema-flexible
Query	SQL	Varies by system
Strength	Complex queries, transactions	Scalability, flexible data, distributed systems
Examples	PostgreSQL, MySQL, Oracle	MongoDB, Redis, Cassandra, Neo4j

Q. When use NoSQL?

A. When data is semi-structured, schema changes frequently, or horizontal scalability is more important than complex relational joins.

8. Machine Learning and Deep Learning - Full Theory and Viva Q&A

8.1 AI vs ML vs DL

Artificial Intelligence is the broad field of making machines perform tasks requiring intelligence. Machine Learning is a subset of AI where systems learn patterns from data. Deep Learning is a subset of ML using neural networks with many layers.

Q. Difference between ML and DL?

A. ML may use algorithms such as linear regression, decision tree, SVM, or random forest and often relies on hand-designed features. DL uses multi-layer neural networks that learn hierarchical features automatically, usually needing more data and computation.

8.2 Types of learning

Type	Data	Goal	Example
Supervised	Labeled data	Predict output from input	Spam classification, house-price regression
Unsupervised	Unlabeled data	Find structure	Clustering customers
Semi-supervised	Few labeled + many unlabeled	Use unlabeled data to improve learning	Pseudo-labeling medical images
Reinforcement	Agent, action, reward	Learn policy	Game playing, robotics

Q. How train semi-supervised model?

A. Use labeled data to train initial model; infer labels for unlabeled data with confidence threshold; retrain with labeled + confident pseudo-labeled data; validate on real labeled validation set. Also mention consistency regularization or label propagation if asked deeper.

Q. Example of clustering?

A. Group customers by buying behavior, cluster documents by topic, segment images, group network traffic patterns.

8.3 Regression vs classification

Regression predicts continuous numeric values. Classification predicts discrete categories/classes. Logistic regression has "regression" in the name but is used for classification because it models class probability.

Task	Output	Loss examples	Metric examples
Regression	Continuous value	MSE, MAE	RMSE, MAE, R^2
Classification	Class/probability	Cross entropy, hinge loss	Accuracy, precision, recall, F1, AUC

Q. House price prediction?

A. Regression because output is continuous price.

Q. Disease positive/negative?

A. Classification because output is class label.

Q. Can age group prediction be classification?

A. Yes, if output is discrete age group such as child/adult/senior.

8.4 Train/validation/test, overfitting, underfitting

Training set is used to fit model. Validation set tunes hyperparameters and checks model choices. Test set estimates final generalization. Overfitting means model learns training data too specifically and performs poorly on unseen data. Underfitting means model is too simple to capture pattern.

Problem	Training performance	Validation/test performance	Fix
Overfitting	Very good	Poor	More data, regularization, dropout, early stopping, simpler model
Underfitting	Poor	Poor	More complex model, better features, train longer, reduce regularization

Q. What is bias-variance tradeoff?

A. High bias causes underfitting; high variance causes overfitting. Good model balances both.

8.5 Neural networks and activation functions

A neural network consists of layers of neurons. Each neuron computes weighted sum plus bias and applies activation. Without non-linear activation, multiple layers collapse into a linear transformation.

Activation	Range	Use	Issue
Sigmoid	0 to 1	Binary output/probability	Vanishing gradient for large values
Tanh	-1 to 1	Older hidden layers/RNNs	Vanishing gradient
ReLU	0 to infinity	Common hidden layers	Dead neurons possible
Leaky ReLU	Negative small slope + positive	Hidden layers	Extra parameter
Softmax	Probabilities sum to 1	Multiclass output	Used with cross entropy

Q. What activation is commonly used in hidden layers?

A. ReLU, because it is computationally simple and reduces vanishing-gradient issues compared to sigmoid/tanh.

Q. What activation for binary classification output?

A. Sigmoid for one output probability, usually with binary cross entropy.

Q. What activation for multiclass classification?

A. Softmax, usually with categorical cross entropy.

8.6 CNN theory

Convolutional Neural Network is designed for grid-like data such as images. It uses convolution layers to detect local patterns. Early layers learn edges/textures, later layers learn parts/objects.

Convolution layer

A convolution layer applies learnable filters/kernels over local regions. The same filter is shared across spatial positions, reducing parameters and enabling translation equivariance.

Input image/feature map $\rightarrow$ convolution filters $\rightarrow$ feature maps

Filter examples: edge detector, texture detector, shape detector

Pooling layer

Pooling reduces spatial size and computation. Max pooling takes maximum value from a region, preserving strongest activation. Average pooling takes average. Pooling helps robustness to small shifts.

Padding and stride

Padding adds border pixels to control output size. Stride controls step size of filter movement. Larger stride reduces output dimension.

Output size = $\text{floor}((N + 2P - F)/S) + 1$

N=input size, P=padding, F=filter size, S=stride

Q. What does convolution layer do?

A. It learns filters that extract local features from input, such as edges in early layers and complex patterns in deeper layers.

Q. What does pooling layer do?

A. It downsamples feature maps, reduces computation, controls overfitting, and provides small translation robustness.

Q. What is flatten layer?

A. It converts multi-dimensional feature maps into a vector before fully connected layers.

Q. Why CNN better than fully connected network for images?

A. CNN uses local connectivity and weight sharing, greatly reducing parameters and exploiting image spatial structure.

8.7 RNN and LSTM

RNN processes sequential data by maintaining hidden state. It is useful for text, speech, time series, and sequence modeling. Basic RNN struggles with long-term dependencies due to vanishing/exploding gradients. LSTM uses gates to control memory flow.

- Input gate: controls what new information enters memory.
- Forget gate: controls what old memory is discarded.
- Output gate: controls what part of memory influences output.
- Cell state: long-term memory path.

Q. Usage of RNN?

A. Language modeling, sentiment analysis, speech recognition, time-series forecasting, sequence labeling.

Q. Why LSTM over vanilla RNN?

A. LSTM handles longer dependencies better using gates and cell state, reducing vanishing-gradient problem.

Q. What is GRU?

A. Gated Recurrent Unit, a simpler gated recurrent model with fewer gates than LSTM.

8.8 Common ML algorithms

Algorithm	Type	Idea
Linear regression	Supervised regression	Fit line/plane minimizing squared error
Logistic regression	Supervised classification	Model class probability using sigmoid/softmax
Decision tree	Classification/regression	Split data by feature tests
Random forest	Ensemble	Many trees with bagging
SVM	Classification/regression	Maximize margin
K-means	Unsupervised clustering	Assign points to nearest centroids
Naive Bayes	Classification	Bayes theorem with independence assumption

Q. K-means steps?

A. Choose k centroids, assign points to nearest centroid, update centroids, repeat until convergence.

Q. What is supervised classification example?

A. Email spam detection: input email features, output spam or not spam.

9. Networking and Security - Full Theory and Viva Q&A

9.1 OSI and TCP/IP models

OSI layer	Core function	Device/protocol examples
7 Application	User-level network services	HTTP, DNS, SMTP
6 Presentation	Format, encryption, compression	TLS conceptually, encoding
5 Session	Session management	RPC/session control
4 Transport	End-to-end delivery	TCP, UDP
3 Network	Routing and IP addressing	IP, router
2 Data Link	Local delivery, MAC addressing	Ethernet, switch, ARP support
1 Physical	Bits/signals	Cable, radio, hub

Q. Switch and router layer?

A. Switch is Layer 2 using MAC addresses. Router is Layer 3 using IP addresses. Multilayer switches can do routing too.

9.2 ARP

Address Resolution Protocol maps IPv4 address to MAC address in a local network. It is necessary because IP is used at network layer but Ethernet delivery inside LAN needs MAC address.

Host A wants to send to IP_B in same LAN.

1. A checks ARP cache.
2. If not found, A broadcasts: Who has IP_B?
3. Host B replies unicast: IP_B is at MAC_B.
4. A caches IP_B -> MAC_B and sends frame to MAC_B.

Q. Is ARP broadcast or unicast?

A. ARP request is broadcast. ARP reply is usually unicast.

Q. What is ARP spoofing?

A. Attacker sends false ARP replies to map victim IP to attacker MAC, enabling man-in-the-middle or traffic interception.

9.3 NAT and PAT

NAT translates private IP addresses to public IP addresses. It helps conserve IPv4 addresses and hides internal network addressing. PAT translates private IP + port to public IP + port, allowing many hosts to share one public IP.

Feature	NAT	PAT
Mapping	IP-to-IP	IP+port to IP+port
Public IP use	May need multiple public IPs	Many private hosts share one public IP
Also called	Network Address Translation	NAT overload
Example	192.168.1.5 -> 103.x.x.x	192.168.1.5:5000 -> 103.x.x.x:62001

Q. PAT vs NAT?

A. PAT is a form of NAT that uses transport-layer port numbers to distinguish multiple internal connections sharing one public IP.

9.4 IPv4 vs IPv6

Feature	IPv4	IPv6
Address size	32-bit	128-bit
Header	20-60 bytes	Fixed base header 40 bytes
Notation	Dotted decimal	Hexadecimal colon notation
Broadcast	Supports broadcast	No broadcast, uses multicast/anycast
Address space	Limited	Very large
NAT need	Common	Less necessary

Q. IPv4 header size?

A. Minimum 20 bytes, maximum 60 bytes due to options.

Q. IPv6 header size?

A. Fixed base header is 40 bytes. Optional information is carried in extension headers.

9.5 DNS and DNS poisoning

DNS maps domain names to IP addresses. DNS poisoning/cache poisoning inserts false DNS records into resolver cache, causing users to be redirected to malicious IP addresses.

- Consequence: phishing websites, malware delivery, MITM, traffic redirection, denial of service.
- Defenses: DNSSEC, source-port randomization, transaction ID randomization, secure resolvers, HTTPS certificate validation.

Q. What problem arises due to DNS poisoning?

A. A user may type a legitimate domain but be sent to attacker-controlled server, enabling credential theft, malware, or MITM.

Q. Does HTTPS help?

A. HTTPS can help because certificate validation may warn if attacker cannot present valid certificate. But users may still be phished through lookalike domains or compromised certificates.

9.6 TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable with ACK/retransmission	Best effort
Ordering	Preserves order	No ordering guarantee
Overhead	Higher	Lower
Examples	HTTP/HTTPS, SSH, FTP	DNS, VoIP, streaming, gaming

Q. Why DNS often uses UDP?

A. DNS queries are small and need low latency. TCP is used for large responses, zone transfers, or when UDP response is truncated.

9.7 Security fundamentals

Concept	Meaning
Confidentiality	Only authorized parties can read data
Integrity	Data is not altered without detection
Availability	System remains accessible
Authentication	Verify identity
Authorization	Verify permission
Non-repudiation	Cannot deny an action later

Q. Encryption vs hashing?

A. Encryption is reversible with a key; hashing is one-way digest. Passwords should be hashed with salt, not encrypted for normal storage.

Q. Symmetric vs asymmetric encryption?

A. Symmetric uses same key for encryption/decryption. Asymmetric uses public/private key pair.

Q. SQL injection?

A. Attacker injects malicious SQL through input. Prevent using parameterized queries/prepared statements and input validation.

10. Digital Logic Design and Computer Architecture

10.1 Combinational vs sequential circuits

Combinational circuit output depends only on current inputs. Sequential circuit output depends on current inputs and stored previous state. Sequential circuits need memory elements like latches/flip-flops.

Circuit type	Depends on	Memory?	Examples
Combinational	Current inputs only	No	Adder, subtractor, multiplexer, decoder, encoder
Sequential	Current inputs + previous state	Yes	Latch, flip-flop, counter, register, FSM

Q. Is adder combinational?

A. Yes. Sum and carry depend only on current inputs.

Q. Is counter sequential?

A. Yes. Next count depends on previous count/state.

10.2 Latches and flip-flops

A latch is level-sensitive: output can change while enable/clock level is active. A flip-flop is edge-triggered: output changes only at clock edge. Flip-flops are widely used in synchronous circuits.

Memory element	Behavior	Use
SR latch	Set/reset memory	Basic storage, but invalid condition issue
D latch	Stores D when enable active	Simple data storage
D flip-flop	Stores D on clock edge	Registers, pipelines
JK flip-flop	Improved SR, toggles at J=K=1	Counters
T flip-flop	Toggles when T=1	Counters

Q. Why SR latch not commonly used?

A. Because S and R both active can be invalid/forbidden and may create unstable/undefined output. D/JK flip-flops are safer.

Q. Latch vs flip-flop?

A. Latch is level-sensitive; flip-flop is edge-triggered.

10.3 Logic gates and universal gates

Basic gates include AND, OR, NOT. NAND and NOR are universal gates because any Boolean function can be built using only NAND or only NOR.

Q. Why NAND is universal?

A. Using NAND, we can build NOT, AND, and OR; therefore any Boolean expression can be implemented.

Q. What is XOR used for?

A. Addition sum bit, parity checking, comparison, cryptographic operations.

10.4 Memory hierarchy

Computer memory is organized as hierarchy: registers, cache, main memory, secondary storage. Higher levels are faster, smaller, and more expensive per bit. Lower levels are larger and slower.

Level	Speed	Size	Example
Register	Fastest	Very small	CPU registers
Cache	Very fast	Small	L1/L2/L3 cache
Main memory	Moderate	GBs	RAM
Secondary storage	Slow	Large	SSD/HDD

Q. What memory elements are used in computers?

A. At circuit level: latches and flip-flops. At CPU level: registers and cache. At system level: RAM, ROM, SSD/HDD.

Q. Why cache is needed?

A. CPU is much faster than main memory. Cache stores recently/frequently used data to reduce average memory access time.

10.5 CPU basics and pipelining

CPU contains control unit, ALU, registers, and buses. Instruction execution generally follows fetch, decode, execute, memory access, write-back. Pipelining overlaps stages of multiple instructions to improve throughput.

Q. What is pipeline hazard?

A. A situation that prevents next instruction from executing in the next cycle. Types: structural, data, control hazards.

Q. Throughput vs latency?

A. Latency is time for one task; throughput is number of tasks completed per unit time. Pipelining improves throughput, not necessarily single-instruction latency.

10.6 Address calculation for arrays/matrices

Arrays are stored linearly in memory. Address calculation depends on row-major or column-major order. C/C++ use row-major storage.

Row-major 0-based:

$\text{Address}(A[i][j]) = \text{Base} + (i * \text{number_of_columns} + j) * \text{element_size}$

Column-major 0-based:

$\text{Address}(A[i][j]) = \text{Base} + (j * \text{number_of_rows} + i) * \text{element_size}$

Q. If base address=1000, int size=4, A has 10 columns, find A[2][3] row-major.

A. Address = $1000 + (2*10 + 3)*4 = 1000 + 92 = 1092$.

Q. Why need number of columns in row-major formula?

A. Because to reach row i, we skip i full rows, each containing number_of_columns elements.

11. Operating Systems - Full Theory and Viva Q&A

11.1 Process and thread

A process is a program in execution with its own address space. A thread is a lightweight execution unit inside a process. Threads of same process share memory, file descriptors, and resources.

Feature	Process	Thread
Address space	Separate	Shared within process
Communication	IPC needed	Shared memory easier
Context switch	More expensive	Less expensive
Failure isolation	Better	One bad thread can crash process

Q. Why threads are lightweight?

A. They share process resources and require less overhead for creation/context switching compared to processes.

11.2 Scheduling

Algorithm	Idea	Pros/cons
FCFS	First come first served	Simple, convoy effect
SJF	Shortest job first	Optimal average waiting if burst known, starvation possible
Round Robin	Time quantum rotation	Good for time-sharing, quantum selection important
Priority	Higher priority first	Starvation possible, aging helps
Multilevel queue	Different queues/classes	Flexible but complex

Q. What is context switch?

A. Saving state of currently running process/thread and loading state of another.

Q. What is starvation?

A. A process waits indefinitely because others keep getting preference.

11.3 Deadlock

Deadlock occurs when processes wait forever for resources held by each other. Four necessary conditions must hold simultaneously.

1. Mutual exclusion: resource cannot be shared.
2. Hold and wait: process holds resource while waiting for another.
3. No preemption: resource cannot be forcibly taken.
4. Circular wait: circular chain of waiting processes.

Q. How to prevent deadlock?

A. Break at least one necessary condition: allow preemption, avoid hold-and-wait, impose resource ordering to break circular wait, or make resources sharable when possible.

Q. Deadlock vs starvation?

A. Deadlock is circular waiting where no one proceeds. Starvation is indefinite waiting due to scheduling/resource policy, while others may proceed.

11.4 Semaphore and mutex

Mutex is a lock for mutual exclusion, usually owned by one thread. Semaphore is a counter controlling access to one or more resources. Binary semaphore is similar to mutex but ownership semantics can differ.

Q. Mutex vs semaphore?

A. Mutex protects a critical section with one owner. Semaphore counts available resources and can allow multiple accesses depending on value.

Q. What is critical section?

A. Part of code accessing shared resource that must not be executed concurrently by multiple threads/processes.

11.5 Memory management

OS manages main memory using techniques such as paging, segmentation, virtual memory, and swapping. Virtual memory gives each process an illusion of large contiguous address space.

- Paging divides virtual memory into pages and physical memory into frames.
- Page table maps virtual pages to physical frames.
- TLB caches recent address translations.
- Page fault occurs when referenced page is not in physical memory.

Q. Paging vs segmentation?

A. Paging uses fixed-size blocks, reducing external fragmentation. Segmentation uses variable-size logical units like code/data/stack, which can suffer external fragmentation.

Q. What is page fault?

A. When a process accesses a virtual page not currently in RAM, OS loads it from disk or handles invalid access.

11.6 IPC

Inter-process communication allows processes to exchange data and synchronize. Methods include pipe, message queue, shared memory, socket, signal, and file-based communication.

Q. Fastest IPC?

A. Shared memory is usually fastest for data transfer because processes access same memory region, but synchronization is needed.

Q. Socket used for what?

A. Communication between processes, possibly on different machines over network.

12. Structured Programming / C Fundamentals

12.1 Data types, operators, conditionals, loops

Structured programming organizes code using sequence, selection, and iteration. C is procedural and close to memory. Know data types, operators, if/switch, loops, functions, arrays, pointers, structs, and dynamic memory.

Loop	Condition checked	Best use
for	Before each iteration	Known iteration count
while	Before each iteration	Unknown count, condition-based
do-while	After each iteration	Must run at least once

Q. Which loop executes at least once?

A. do-while, because condition is checked after body.

Q. switch vs if-else?

A. switch is suitable for discrete constant equality checks; if-else is more general and supports ranges/complex conditions.

12.2 Function and recursion

A function is a reusable block of code. Parameters pass information into a function. Recursion occurs when a function calls itself. Recursion needs base case and progress toward base case.

Q. Call by value vs call by reference?

A. Call by value passes copy, so original does not change. Call by reference/pointer passes access to original, so function can modify it.

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Q. Why recursion can cause stack overflow?

A. Each recursive call uses stack space. If recursion depth is too high or base case missing, stack memory exhausts.

12.3 Pointers and arrays

A pointer stores memory address. Arrays and pointers are closely related in C, but not identical. Array name often decays to pointer to first element when passed to function.

```
int x = 10;
int *p = &x;
printf("%d", *p); // dereference, prints 10
```

Q. What is null pointer?

A. A pointer that points to no valid object. Dereferencing null pointer causes undefined behavior/crash.

Q. Pointer vs array?

A. Array is a block of elements; pointer is a variable storing an address. In many expressions array name decays to pointer to first element.

12.4 Struct, union, enum

struct groups different variables under one name. union shares memory among members, so only one member is meaningful at a time. enum defines named integer constants.

Q. struct vs union?

A. In struct, each member has separate storage. In union, all members share same memory; size is at least size of largest member.

12.5 Dynamic memory

Dynamic memory allocation uses heap memory at runtime. In C: malloc, calloc, realloc, free. In C++: new/delete.

Function	Meaning
----------	---------

malloc	Allocates uninitialized memory
calloc	Allocates zero-initialized memory
realloc	Changes allocated block size
free	Releases allocated memory

Q. malloc vs calloc?

A. malloc allocates uninitialized memory. calloc initializes allocated memory to zero and takes number of elements and element size.

Q. What is memory leak?

A. Allocated memory that is no longer reachable and not freed.

12.6 Bitwise and 2s complement

Bitwise operators operate on bits: AND &, OR |, XOR ^, NOT ~, left shift <<, right shift >>. 2s complement represents signed integers.

Q. How to get 2s complement?

A. Invert all bits and add 1.

Q. Why use 2s complement?

A. It simplifies hardware arithmetic because addition/subtraction of signed integers works uniformly.

13. Software Engineering, SOLID, and Design Patterns

13.1 SDLC

Software Development Life Cycle includes requirement analysis, design, implementation, testing, deployment, and maintenance. Models include waterfall, iterative, spiral, and agile.

Model	Idea	When useful
Waterfall	Sequential phases	Stable requirements
Agile	Iterative/incremental	Changing requirements
Spiral	Risk-driven iterations	Large high-risk projects
Prototype	Build prototype to clarify requirements	Unclear requirements

Q. Agile vs waterfall?

A. Waterfall is sequential and plan-driven. Agile is iterative, adaptive, and emphasizes frequent feedback.

13.2 Requirements and testing

Functional requirements describe what the system does. Non-functional requirements describe quality constraints such as performance, security, reliability, usability.

Test	Meaning
Unit test	Tests individual function/module
Integration test	Tests interaction between modules
System test	Tests complete system
Acceptance test	Checks user/customer acceptance
Regression test	Ensures changes did not break old behavior

Q. Verification vs validation?

A. Verification: are we building the product right? Validation: are we building the right product?

13.3 SOLID principles

Principle	Meaning
Single Responsibility	A class should have one reason to change
Open/Closed	Open for extension, closed for modification
Liskov Substitution	Child should be substitutable for parent
Interface Segregation	Prefer small specific interfaces
Dependency Inversion	Depend on abstractions, not concrete classes

Q. Explain Liskov substitution simply.

A. If code works with a base class, it should also work correctly with any subclass without unexpected behavior.

13.4 Design patterns

Pattern	Type	Purpose
Singleton	Creational	Ensure only one instance
Factory	Creational	Create objects without exposing exact class
Observer	Behavioral	Notify dependents when state changes
Strategy	Behavioral	Select algorithm at runtime
Adapter	Structural	Make incompatible interfaces work together
Decorator	Structural	Add behavior without changing class

Q. Singleton drawback?

A. Can introduce global state, make testing difficult, and hide dependencies.

Q. Strategy vs factory?

A. Strategy chooses behavior/algorithm; factory chooses object creation logic.

14. Extra Core CSE Topics That May Appear

14.1 Compiler basics

A compiler translates source code into target code. Main phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, code generation.

Phase	Function
Lexical analysis	Converts characters to tokens
Syntax analysis	Builds parse tree and checks grammar
Semantic analysis	Checks type/scope/meaning
IR generation	Creates intermediate representation
Optimization	Improves code
Code generation	Produces target code

Q. Compiler vs interpreter?

A. Compiler translates before execution; interpreter executes line by line or statement by statement. Many modern systems combine both.

14.2 Automata basics

Finite automata recognize regular languages. DFA has exactly one transition for each state-symbol pair. NFA may have multiple transitions or epsilon transitions.

Q. DFA vs NFA?

A. DFA has deterministic next state. NFA may have multiple possible next states. Both recognize regular languages.

14.3 Numerical/common math basics

Sometimes panels ask simple logic/math to check reasoning. Be careful with exact conditions.

- Modulo: $a \bmod b$ is remainder after integer division.
- Prime number: integer greater than 1 with exactly two positive divisors.
- Logarithm: inverse of exponentiation.
- Geometric series: $1+2+4+\dots+2^h = 2^{h+1}-1$.

15. Master Probable Viva Question Bank

Use this after reading theory. Try to answer each question in 30-60 seconds.

OOP

- Define class and object.
- Why use encapsulation?
- private vs protected.
- Can private member be accessed?
- Function overloading vs overriding.
- Why not overload by return type?
- What is virtual function?
- Friend function use and problem.
- Diamond problem and solution.
- Interface vs abstract class.
- Constructor vs destructor.
- Static member function limitation.

DSA/Algorithms

- Array vs linked list.
- Stack vs queue.
- Find middle of linked list.
- Detect cycle in linked list.
- Types of binary tree.
- BST search complexity.
- Nodes in perfect binary tree height h.
- Hash table $O(1)$ caveat.
- BFS vs DFS.
- Depth-limited search.
- Prim vs Kruskal.
- Dijkstra negative edge problem.
- DP vs greedy.
- GCD recursion.
- Quotient/remainder without division/mod.

DBMS

- Primary vs foreign key.
- Candidate key vs super key.
- Normalization 1NF/2NF/3NF/BCNF.
- ACID with example.
- Dirty read/lost update.
- WHERE vs HAVING.
- Index pros and cons.
- B+ tree index.
- SQL vs NoSQL.
- Join types.

ML/DL

- Regression vs classification.
- ML vs DL.
- Supervised vs unsupervised vs semi-supervised.
- How to train with few labeled + many unlabeled data.

- Example of clustering.
- Overfitting and fixes.
- Bias-variance tradeoff.
- CNN convolution layer.
- Pooling layer.
- Activation functions.
- RNN and LSTM usage.
- Why ReLU common.

Networking/Security

- ARP process.
- Switch vs router layer.
- NAT vs PAT.
- IPv4 vs IPv6 header size.
- DNS poisoning consequences.
- TCP vs UDP.
- OSI layers.
- ARP spoofing.
- Encryption vs hashing.
- Authentication vs authorization.
- SQL injection prevention.

DLD/Architecture

- Combinational vs sequential.
- What is latch?
- SR latch problem.
- D latch vs D flip-flop.
- JK/T flip-flop.
- Memory elements in computer.
- Memory hierarchy.
- Cache purpose.
- Pipeline hazards.
- Row-major address formula.

OS

- Process vs thread.
- Context switch.
- Deadlock conditions.
- Mutex vs semaphore.
- Paging vs segmentation.
- Virtual memory.
- Page fault.
- Scheduling algorithms.
- IPC methods.

SPL/C

- Call by value vs reference.
- Pointer basics.
- Array and pointer difference.
- malloc vs calloc.
- Memory leak.

- struct vs union.
- for vs while vs do-while.
- 2s complement.
- Bitwise operators.

HR/Thesis

- Why teacher?
- Why UIU?
- Higher studies plan.
- Tell us about thesis.
- Why LLM for security?
- Limitations of LLM-based analysis.
- What is responsible disclosure?
- What courses can you teach?
- How to handle weak students?

16. Final Rapid Formula and Trap Sheet

Topic	Formula / Direct answer	Trap
2 green from 4 red + 6 green	$4 + 2 = 6$	Worst case all red first
2 aces from deck	$48 + 2 = 50$	Worst case all non-aces first
3 red cards from deck	$26 + 3 = 29$	Worst case all black first
Perfect tree nodes root height 0	$2^{(h+1)}-1$	Ask height convention
Perfect tree nodes root level 1	$2^h - 1$	Do not say only about 2^h
Row-major address	Base + $(i \cdot \text{cols} + j) \cdot \text{size}$	0-based; adjust lower bound if not 0
Column-major address	Base + $(j \cdot \text{rows} + i) \cdot \text{size}$	Used in Fortran/MATLAB style
IPv4 header	20-60 bytes	Options make it variable
IPv6 base header	40 bytes	Extension headers separate
Switch	Layer 2	Multilayer switch exception
Router	Layer 3	Uses IP addresses
PAT	NAT using ports	Many private hosts share one public IP
Dijkstra	No negative weights	May seem correct in some cases but not guaranteed
Prim dense graph	$O(V^2)$ with matrix	Good for dense
Kruskal	$O(E \log E)$	Good for sparse/edge list
GCD	$\text{gcd}(a,b)=\text{gcd}(b,a\%b)$	Base case $b=0$
ReLU	Common hidden activation	Dead ReLU possible
Softmax	Multiclass output	Probabilities sum to 1
Sigmoid	Binary output	Vanishing gradient in deep hidden layers

17. Final Advice for the Viva Room

- Answer the exact question first. Do not begin with a long story.
- When a formula depends on convention, state the convention: "If root height is 0...".
- For paper questions, write the worst-case arrangement or formula first.
- If asked favorite courses, choose subjects you can defend: DSA, Algorithms, OOP, DBMS, ML.
- Do not overclaim thesis results. Emphasize validation and responsible analysis.
- For ML/DL questions, give both definition and example.
- For networking questions, draw packet/process flow: ARP request/reply, NAT mapping, DNS resolution.
- For DLD questions, distinguish clearly: combinational has no memory; sequential has memory.
- If unsure, be honest but show partial reasoning.
- Keep voice calm. Panels often test confidence and clarity more than rare details.